AIAC LAB 5.1 & 6

NAME : - CH.NAKSHATRA

HALL TICKET NO :- 2303A51094

BATCH - 02

Task 1:

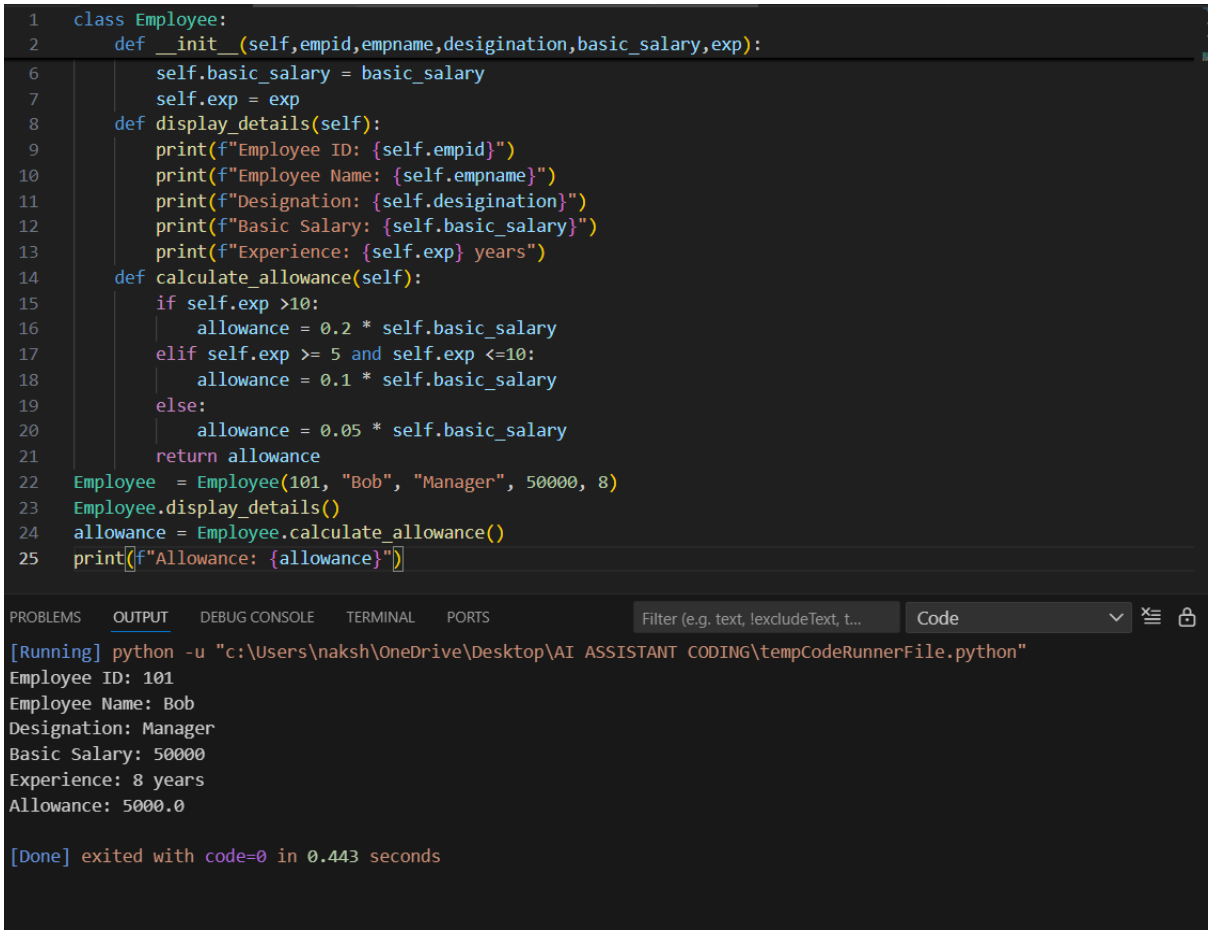
Employee Data: Create Python code that defines a class named `Employee` with the following attributes: `empid`, `empname`, `designation`, `basic_salary`, and `exp`. Implement a method `display_details()` to print all employee details. Implement another method `calculate_allowance()` to determine additional allowance based on experience:

- If `exp > 10 years` → allowance = 20% of `basic_salary`

- If `5 ≤ exp ≤ 10 years` → allowance = 10% of `basic_salary`

- If `exp < 5 years` → allowance = 5% of `basic_salary`

Finally, create at least one instance of the `Employee` class, call the `display_details()` method, and print the calculated allowance



```
class Employee:
    def __init__(self,empid,empname,desigination,basic_salary,exp):
        self.basic_salary = basic_salary
        self.exp = exp
    def display_details(self):
        print(f"Employee ID: {self.empid}")
        print(f"Employee Name: {self.empname}")
        print(f"Designation: {self.desigination}")
        print(f"Basic Salary: {self.basic_salary}")
        print(f"Experience: {self.exp} years")
    def calculate_allowance(self):
        if self.exp >10:
            allowance = 0.2 * self.basic_salary
        elif self.exp >= 5 and self.exp <=10:
            allowance = 0.1 * self.basic_salary
        else:
            allowance = 0.05 * self.basic_salary
        return allowance
Employee  = Employee(101, "Bob", "Manager", 50000, 8)
Employee.display_details()
allowance = Employee.calculate_allowance()
print(f"Allowance: {allowance}")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS   Filter (e.g. text, !excludeText, t...   Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\AI ASSISTANT CODING\tempCodeRunnerFile.python"
Employee ID: 101
Employee Name: Bob
Designation: Manager
Basic Salary: 50000
Experience: 8 years
Allowance: 5000.0

[Done] exited with code=0 in 0.443 seconds
```

Task 2:

Electricity Bill Calculation- Create Python code that defines a class

named `ElectricityBill` with attributes: `customer_id`, `name`, and

`units_consumed`. Implement a method `display_details()` to print

customer details, and a method `calculate_bill()` where:

- Units ≤ 100 → ₹5 per unit

- 101 to 300 units → ₹7 per unit

- More than 300 units → ₹10 per unit

Create a bill object, display details, and print the total bill amount.

```python
class ElectricityBill:
    def display_details(self):
        print(f"Customer ID: {self.customer_id}")
        print(f"Customer Name: {self.name}")
        print(f"Units Consumed: {self.units_consumed}")
    def calculate_bill(self):
        if self.units_consumed <= 100:
            bill_amount = self.units_consumed * 5
        elif self.units_consumed <= 300:
            bill_amount = (100 * 5) + (self.units_consumed - 100) * 7
        else:
            bill_amount = (100 * 5) + (100 * 7) + (self.units_consumed - 200) * 10
        return bill_amount

customer = ElectricityBill(1, "lalith" ,250)
customer.display_details()
bill = customer.calculate_bill()
print(f"Total Bill Amount: {bill}")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS   Filter (e.g. text, !excludeText, t...   Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\AI ASSISTANT CODING\tempCodeRunnerFile.python"
Customer ID: 1
Customer Name: lalith
Units Consumed: 250
Total Bill Amount: 1550

[Done] exited with code=0 in 0.392 seconds
```

Task 3:

Product Discount Calculation- Create Python code that defines a class named `Product` with attributes: `product_id`, `product_name`,

`price`, and `category`. Implement a method `display_details()` to print product details. Implement another method

`calculate_discount()` where:

- Electronics → 10% discount

- Clothing → 15% discount

- Grocery → 5% discount

Create at least one product object, display details, and print the final

price after discount

```python
class Product:
    def __init__(self,product_id,product_name,price,category):
        self.product_id = product_id
        self.product_name = product_name
        self.price = price
        self.category = category
    def display_details(self):
        print(f"Product ID: {self.product_id}")
        print(f"Product Name: {self.product_name}")
        print(f"Price: {self.price}")
        print(f"Category: {self.category}")
    def calculate_discount(self):
        if self.category.lower() == "electronics":
            discount = 0.1 * self.price
        elif self.category.lower() == "clothing":
            discount = 0.15 * self.price
        else:
            discount = 0.05 * self.price
        return discount

product = Product(201, "Smartphone", 20000, "Electronics")
product.display_details()
discount = product.calculate_discount()
print(f"Discount: {discount}")
print(f"Price after discount: {product.price - discount}")
```

```
[Running] python -u "c:\Users\naksh\OneDrive\Desktop\AI ASSISTANT CODING\tempCodeRunnerFile.python"
Product ID: 201
Product Name: Smartphone
Price: 20000
Category: Electronics
Discount: 2000.0
Price after discount: 18000.0

[Done] exited with code=0 in 0.434 seconds
```

Task 4:

Book Late Fee Calculation- Create Python code that defines a class

named `LibraryBook` with attributes: `book_id`, `title`, `author`,

`borrower`, and `days_late`. Implement a method `display_details()`

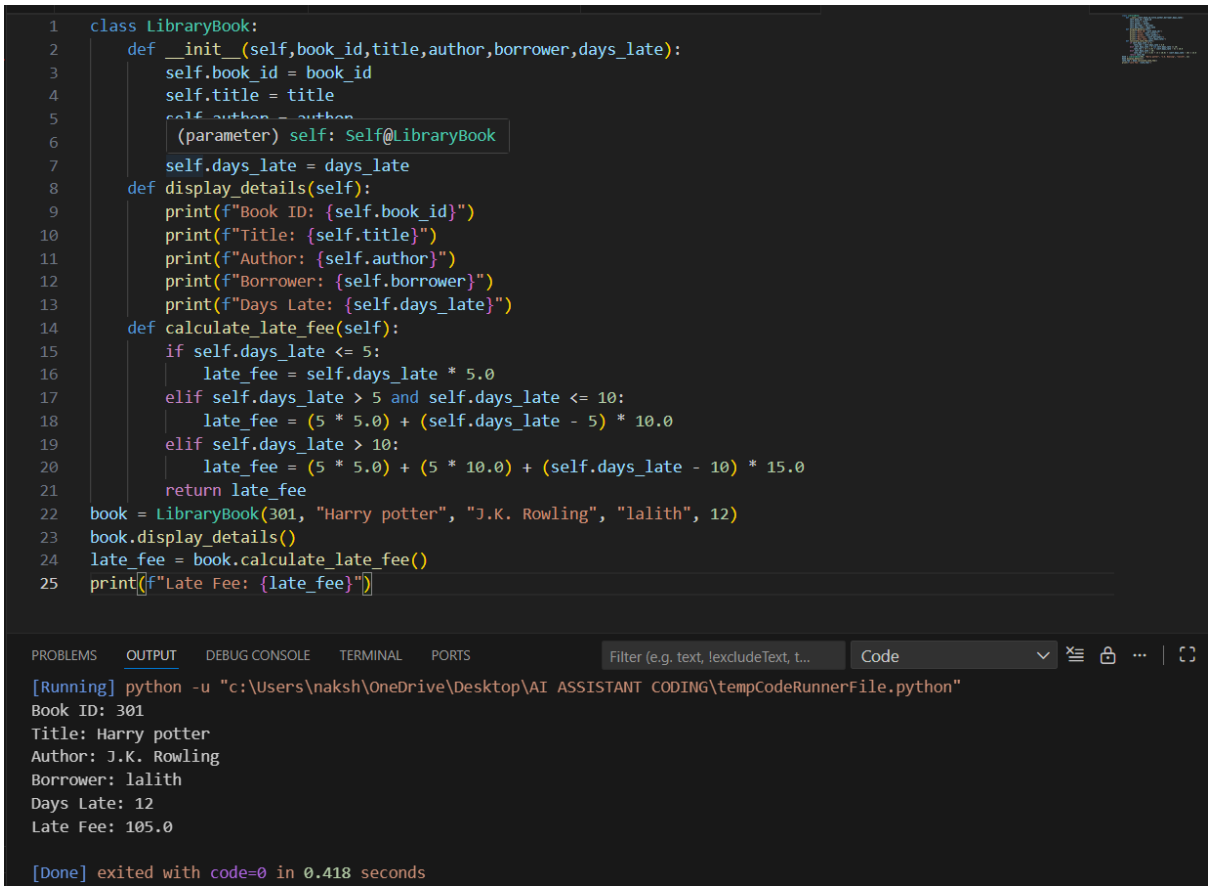
to print book details, and a method `calculate_late_fee()` where:

- Days late ≤ 5 → ₹5 per day

- 6 to 10 days late → ₹7 per day

- More than 10 days late → ₹10 per day

Create a book object, display details, and print the late fee.



```python
class LibraryBook:
    def __init__(self,book_id,title,author,borrower,days_late):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.borrower = borrower
        self.days_late = days_late
    def display_details(self):
        print(f"Book ID: {self.book_id}")
        print(f"Title: {self.title}")
        print(f"Author: {self.author}")
        print(f"Borrower: {self.borrower}")
        print(f"Days Late: {self.days_late}")
    def calculate_late_fee(self):
        if self.days_late <= 5:
            late_fee = self.days_late * 5.0
        elif self.days_late > 5 and self.days_late <= 10:
            late_fee = (5 * 5.0) + (self.days_late - 5) * 10.0
        elif self.days_late > 10:
            late_fee = (5 * 5.0) + (5 * 10.0) + (self.days_late - 10) * 15.0
        return late_fee
book = LibraryBook(301, "Harry potter", "J.K. Rowling", "lalith", 12)
book.display_details()
late_fee = book.calculate_late_fee()
print(f"Late Fee: {late_fee}")
```

(parameter) self: Self@LibraryBook

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, !excludeText, t... Code

```
[Running] python -u "c:\Users\naksh\OneDrive\Desktop\AI ASSISTANT CODING\tempCodeRunnerFile.python"
Book ID: 301
Title: Harry potter
Author: J.K. Rowling
Borrower: lalith
Days Late: 12
Late Fee: 105.0

[Done] exited with code=0 in 0.418 seconds
```

Task 5:

Student Performance Report - Define a function

`student_report(student_data)` that accepts a dictionary containing

student names and their marks. The function should:

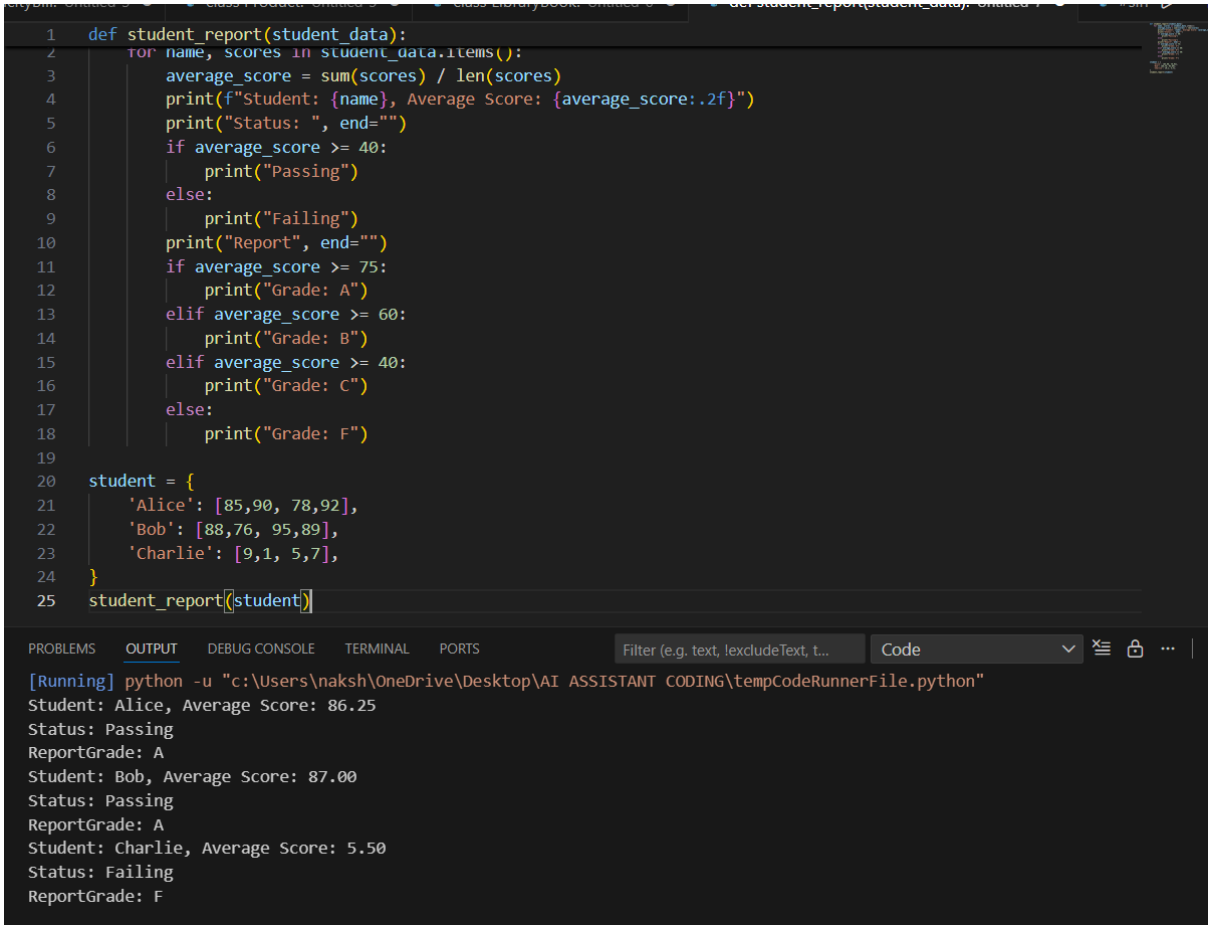
- Calculate the average score for each student

- Determine pass/fail status (pass ≥ 40)

- Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the

output.



```
def student_report(student_data):
    for name, scores in student_data.items():
        average_score = sum(scores) / len(scores)
        print(f"Student: {name}, Average Score: {average_score:.2f}")
        print("Status: ", end="")
        if average_score >= 40:
            print("Passing")
        else:
            print("Failing")
        print("Report", end="")
        if average_score >= 75:
            print("Grade: A")
        elif average_score >= 60:
            print("Grade: B")
        elif average_score >= 40:
            print("Grade: C")
        else:
            print("Grade: F")

student = {
    'Alice': [85,90, 78,92],
    'Bob': [88,76, 95,89],
    'Charlie': [9,1, 5,7],
}
student_report(student)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS   Filter (e.g. text, !excludeText, t...   Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\AI ASSISTANT CODING\tempCodeRunnerFile.python"
Student: Alice, Average Score: 86.25
Status: Passing
ReportGrade: A
Student: Bob, Average Score: 87.00
Status: Passing
ReportGrade: A
Student: Charlie, Average Score: 5.50
Status: Failing
ReportGrade: F
```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named

`TaxiRide` with attributes: `ride_id`, `driver_name`, `distance_km`,

and `waiting_time_min`. Implement a method `display_details()` to

print ride details, and a method `calculate_fare()` where:

- ₹15 per km for the first 10 km

- ₹12 per km for the next 20 km

- ₹10 per km above 30 km

- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

```python
class TaxiRide:
    def __init__(self,ride_id,driver_name,distance_km,waiting_time_min):
        self.ride_id = ride_id
        self.driver_name = driver_name
        self.distance_km = distance_km
        self.waiting_time_min = waiting_time_min
    def display_details(self):
        print(f"Ride ID: {self.ride_id}")
        print(f"Driver Name: {self.driver_name}")
        print(f"Distance (km): {self.distance_km}")
        print(f"Waiting Time (min): {self.waiting_time_min}")
    def calculate_fare(self):
        if self.distance_km <= 10:
            fare = self.distance_km * 15.0 + self.waiting_time_min * 2.0
        elif self.distance_km > 10 and self.distance_km <= 30:
            fare = (10 * 15.0) + (self.distance_km - 10) * 12.0 + self.waiting_time_min * 2.0
        else:
            fare = (10 * 15.0) + (20 * 12.0) + (self.distance_km - 30) * 10.0 + self.waiting_time_min * 2.0
        return fare
ride = TaxiRide(401, "jhonny", 35, 15)
ride.display_details()
fare = ride.calculate_fare()
print(f"Total Fare: {fare}")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS   Filter (e.g. text, !excludeText, t...   Code

[Running] python -u "c:\Users\naksh\OneDrive\Desktop\AI ASSISTANT CODING\tempCodeRunnerFile.python"
Ride ID: 401
Driver Name: jhonny
Distance (km): 35
Waiting Time (min): 15
Total Fare: 470.0

[Done] exited with code=0 in 0.445 seconds
```

Task 7:

Statistics Subject Performance - Create a Python function

`statistics_subject(scores_list)` that accepts a list of 60 student scores

and computes key performance statistics. The function should return

the following:

- Highest score in the class

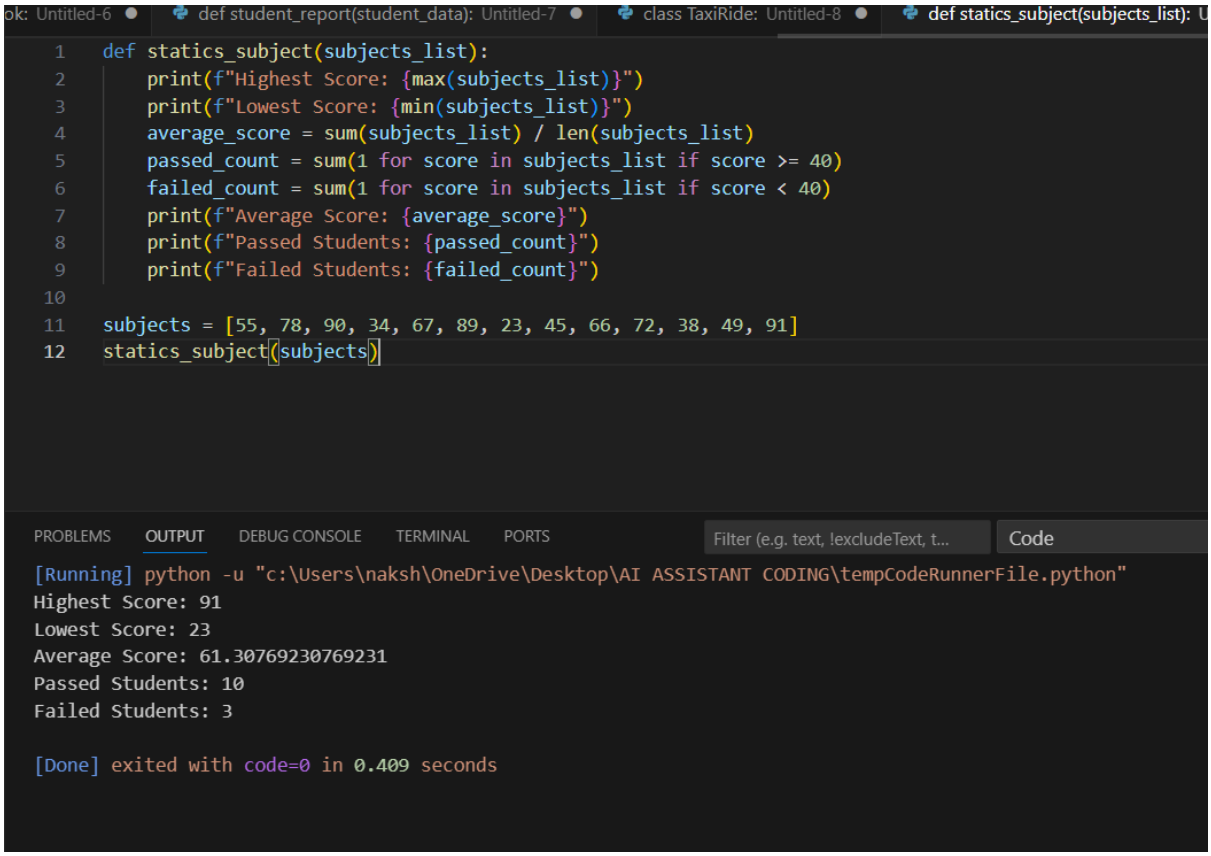
- Lowest score in the class

- Class average score

- Number of students passed (score ≥ 40)

- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic



Task Description #8 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

• Naive approach(basic)

• Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain

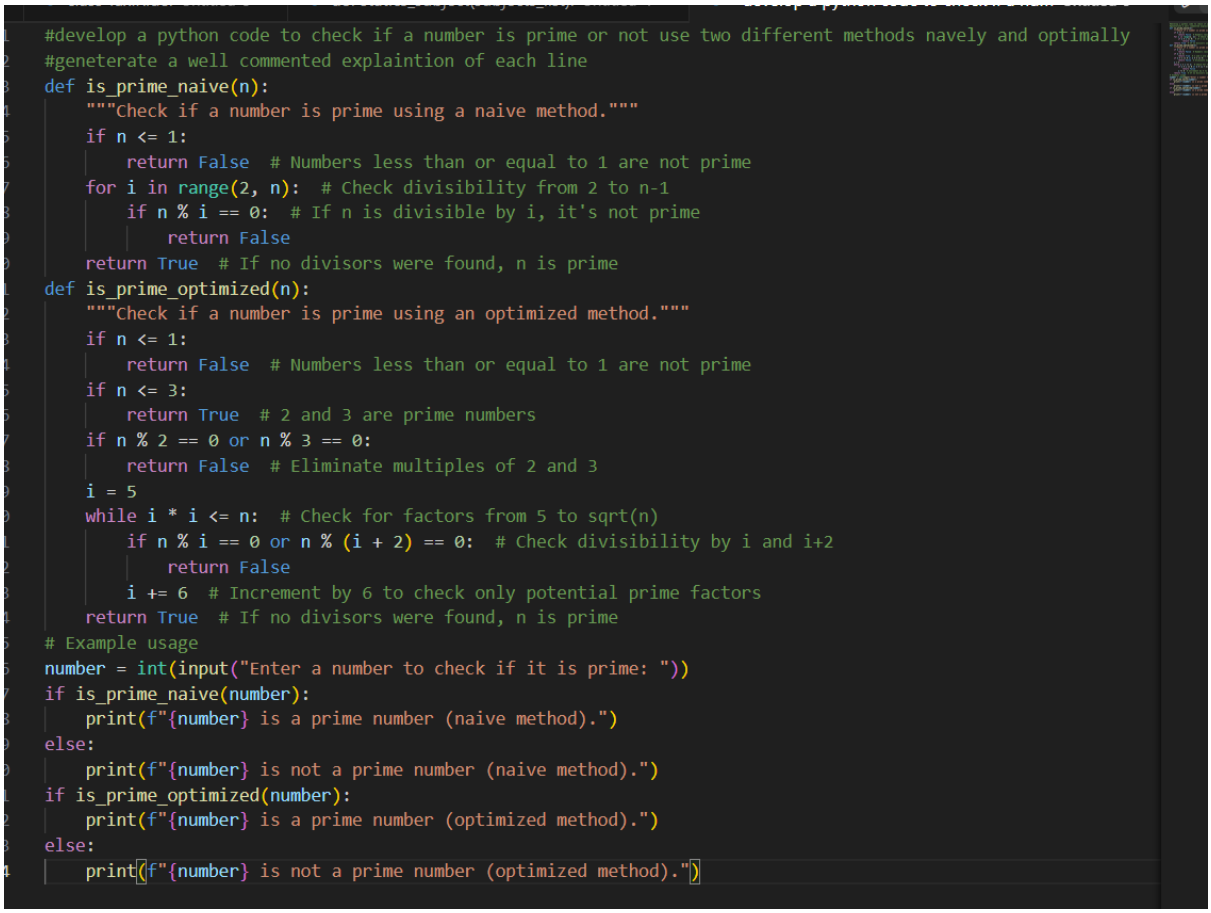
how the optimized version improves performance.”

Expected Output:

• Code for both methods.

• Transparent explanation of time complexity.

• Comparison highlighting efficiency improvements.



```python
#develop a python code to check if a number is prime or not use two different methods navely and optimally
#geneterate a well commented explaintion of each line
def is_prime_naive(n):
    """Check if a number is prime using a naive method."""
    if n <= 1:
        return False  # Numbers less than or equal to 1 are not prime
    for i in range(2, n):  # Check divisibility from 2 to n-1
        if n % i == 0:  # If n is divisible by i, it's not prime
            return False
    return True  # If no divisors were found, n is prime
def is_prime_optimized(n):
    """Check if a number is prime using an optimized method."""
    if n <= 1:
        return False  # Numbers less than or equal to 1 are not prime
    if n <= 3:
        return True  # 2 and 3 are prime numbers
    if n % 2 == 0 or n % 3 == 0:
        return False  # Eliminate multiples of 2 and 3
    i = 5
    while i * i <= n:  # Check for factors from 5 to sqrt(n)
        if n % i == 0 or n % (i + 2) == 0:  # Check divisibility by i and i+2
            return False
        i += 6  # Increment by 6 to check only potential prime factors
    return True  # If no divisors were found, n is prime
# Example usage
number = int(input("Enter a number to check if it is prime: "))
if is_prime_naive(number):
    print(f"{number} is a prime number (naive method).")
else:
    print(f"{number} is not a prime number (naive method).")
if is_prime_optimized(number):
    print(f"{number} is a prime number (optimized method).")
else:
    print(f"{number} is not a prime number (optimized method).")
```

Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.

2. Ask AI to explain base cases and recursive calls.

Expected Output:

• Well-commented recursive code.

• Clear explanation of how recursion works.

• Verification that explanation matches actual execution.

```python
#generate python code for fibannaci series using recursion
#generate a well commented explanation of each line soo that a every step can be understood
def fibonacci(n):
    """Return the nth Fibonacci number using recursion."""
    if n <= 0:
        return 0  # The 0th Fibonacci number is 0
    elif n == 1:
        return 1  # The 1st Fibonacci number is 1
    else:
        # Recursively call fibonacci for the two preceding numbers
        return fibonacci(n - 1) + fibonacci(n - 2)
        # The nth Fibonacci number is the sum of the (n-1)th and (n-2)th Fibonacci numbers
# Example usage
terms = int(input("Enter the number of terms in the Fibonacci series: "))
print("Fibonacci Series:")
for i in range(terms):
    print(fibonacci(i), end=" ")  # Print each Fibonacci number in the series
```

Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for

each exception.”

Expected Output:

• Code with meaningful exception handling.

• Clear comments explaining each error scenario.

• Validation that explanations align with runtime behavior.

```python
'''Generate code with proper error handling and clear explanations for each exception
with meaningful exception handling and Clear comments explaining each error scenario.'''
def divide_numbers(a, b):
    """Divide two numbers with error handling."""
    try:
        result = a / b  # Attempt to divide a by b
    except ZeroDivisionError:
        # Handle the case where b is zero
        print("Error: Cannot divide by zero. Please provide a non-zero denominator.")
        return None
    except TypeError:
        # Handle the case where inputs are not numbers
        print("Error: Invalid input type. Please provide numeric values for both numerator and denominator.")
        return None
    else:
        # If no exceptions occur, return the result
        return result
# Example usage
num1 = 10
num2 = 0  # Change this value to test different scenarios
division_result = divide_numbers(num1, num2)
print(f"Division Result: {division_result}")
```